ChatGPT

Prompts for Generating the MississippiFM Codebase

The MississippiFM project extends the existing `poultry_geo` package by introducing overlap-aware self-supervised learning on sub-meter aerial imagery, integrating a DINOv3 Vision Transformer backbone, and expanding the evaluation to multiple geospatial tasks. The following prompts are meant to guide the generation of a comprehensive codebase that implements this vision.

1. Adjacency Index and Tile Metadata

Write a Python script called `create_adjacency_index.py` that constructs a lightweight adjacency index for existing NAIP tiles. The script should:

- Accept a root directory containing millions of 512×512 JPEG tiles where each filename encodes the original raster name and the x/y tile offsets.
- Parse filenames to recover the raster identifier and the tile's starting row and column indices in pixel space.
- Identify all pairs of tiles that overlap by 50 % in both x and y directions. For each overlapping pair, compute the pixel offsets that map the shared sub-region from the first tile into the second.
- Output a simple CSV file where each record contains: `tile_a`, `tile_b`, `row_offset`, `col_offset`, `overlap_x1`, `overlap_y1`, `overlap_x2`, `overlap_y2`. Use integer pixel indices throughout. Do not reslice the source imagery; use only filename metadata.
- Provide command-line arguments for the input directory, the overlap ratio, and the output path.

2. Overlap-Aware Dataset Class

Implement a PyTorch `Dataset` class named `OverlapAwareNAIPDataset` that extends the existing `NAIPDataset` to support overlap-aware self-supervised learning. This dataset should:

- Read the adjacency index created above and load tile pairs on demand. In each epoch, sample three types of pairs: 50 % overlapping adjacent tiles, 25 % two independent augmentations of the same tile, and 25 % independent random tiles.
- For overlapping pairs, return both images along with tensors describing the overlap mask coordinates. Compute and return the set of patch indices that correspond to the shared geographic region given the 16×16 patch grid size of ViT-B/16.
- For each sample, apply synchronized random augmentations (color jitter, flips, crops) while maintaining the integrity of the pixel offsets so that the overlap mapping remains accurate.
- Support optional high-resolution adaptation: when `high_res=True`, produce 512×512 crops with recorded crop origins; otherwise use random 256×256 or 320×320 views.
- Include methods to preload tile metadata and to switch between frozen and train splits. Do not allow adjacent tiles from the same raster to appear in both train and validation partitions.

3. DINOv3-Based MississippiFM Model

Create a PyTorch module called `MississippiFMEncoder` that wraps a DINOv3 ViT-B/16 backbone (from `timm` or a similar library) and adds heads for the four training objectives. The module should:

- Expose a method `forward_views` that accepts a batch of global and local views, returns the CLS token embeddings and patch embeddings, and projects them through separate MLP heads for global distillation and masked patch prediction.
- Include an MLP projection head used exclusively for the geographic overlap alignment loss. This head should map patch embeddings into a lower-dimensional space for cosine-similarity alignment.
- Implement a Gram anchoring mechanism: during initialization, load the weights of a pretrained DINOv3 model and freeze a copy as the anchor. During training, compute the Gram matrices of current patch embeddings and anchor embeddings and penalize deviations via a mean-squared-error loss scaled by a tunable weight.
- Provide methods to enable or disable LoRA adapters and NF4 quantization for parameter-efficient fine-tuning.

4. Training Loop with Multiple Objectives

Write a training script (`train_mississippifm.py`) that orchestrates the joint optimization of the four objectives: global self-distillation, masked patch prediction, geographic overlap alignment, and Gram anchoring. The script should:

- Accept configuration parameters from a YAML file specifying learning rates, batch sizes, loss weights, and the proportions of overlapping vs. independent pairs.
- Use the `OverlapAwareNAIPDataset` to produce batches containing both overlapping and independent pairs. For each batch, compute the four losses: DINO-style global distillation between different views, iBOT-style masked patch prediction, cosine alignment between corresponding patches of overlapping tiles, and Gram anchoring against the frozen backbone.
- Perform two-stage training: first train on random 256×256 or 320×320 crops for the majority of epochs, then adapt to full 512×512 crops for the final 10 %–15 % of training. Use mixed precision (`bf16` or `fp16`) and checkpoint optimizer and scheduler states after every epoch to allow resumption.
- Log training and validation metrics (loss components, learning rate, gradient norms) to TensorBoard and write a command log similar to the existing CLI wrappers.
- Save checkpoints at regular intervals and provide a `--resume` option to continue training from the last checkpoint. Include code to evaluate frozen representation quality on downstream validation probes (e.g., poultry segmentation) to select the best checkpoint.

5. Downstream Evaluation and Probes

Extend the current evaluation suite to support the diagnostic probes and cross-task benchmarks described in the MississippiFM specification. Implement:

- A script `evaluate_probes.py` that loads a frozen MississippiFM encoder, attaches simple linear or shallow heads, and measures performance on: agricultural pattern segmentation

(Agriculture-Vision), settlement context classification (OpenEarthMap or similar), building segmentation (Inria), oriented object detection (DOTA), and poultry-house segmentation (Alabama/Georgia). Use standard metrics like mIoU, Dice, AP50:95, and boundary F1.
- Utilities to run zero-shot geographic transfer evaluations: apply Alabama-trained thresholds to Georgia data without retraining and compute performance drops. Support optional few-shot fine-tuning on a small set of Georgia farms.
- Diagnostic scripts that compute high-level spatial probes: building density regression, road-building relationship classification, field structure prediction, texture retrieval, and context augmentation experiments. These should operate on frozen features and report metrics like MAE, RMSE, recall@k, and F1.
- A module `geographic_benchmark.py` that orchestrates held-out county splits, different ground sampling distances, and temporal splits to produce the strict geographic and scale-shift benchmark results.

6. CLI Integration and Packaging

Update the existing `poultry_geo` CLI to include commands for MississippiFM pretraining and evaluation. Add a `mississippi-pretrain` command that wraps `train_mississippifm.py`, accepting options for data directories, adjacency index, model size (`vit_s`, `vit_b`, `vit_l`), and loss weights. Add `mississippi-eval` commands that invoke the probe and benchmark scripts. Ensure that each command logs arguments, supports resumption, and writes outputs to predictable directory structures under `trained_models/mississippifm/`.

Provide proper setup in `setup.py` or `pyproject.toml` to install dependencies (`timm`, `scikit-learn`, `openmmlab`, etc.) and expose console entry points. Add unit tests to the existing PyTest suite to cover adjacency index creation, overlap mapping logic, and loss computation correctness.

7. High-Level Project Guidance

Include a `README.md` describing the motivation for MississippiFM, the overlap-aware self-supervised approach, installation steps, data preparation, training commands, evaluation procedures, and citation guidelines. Emphasize that the existing six-million-tile corpus should not be regenerated and that the adjacency index provides the only additional metadata required for overlap training.